Name: Ashwin Rai Roll No. : 202403069

Practical No. 5: Key Exchange using Diffie-Hellman

Aim

To study and implement the **Diffie-Hellman Key Exchange algorithm** for securely establishing a shared secret key between two entities over an insecure communication channel.

Objectives

- To understand the concept of Key Exchange.
- To study Symmetric and Asymmetric Key Exchange techniques.
- To understand the working of the Diffie-Hellman algorithm.
- To generate public and private keys for two entities.
- To calculate a common shared secret key between two entities.
- To understand how the shared key can be used for secure communication.

Theory

1. Key Exchange

- **Key Exchange** is a fundamental concept in cryptography that allows two parties to securely establish a shared secret key over an insecure communication channel.
- The shared secret key can then be used with symmetric encryption algorithms for secure communication.
- One of the widely studied key exchange algorithms is the **Diffie-Hellman algorithm**.

2. Key Exchange Techniques

Key exchange techniques can be broadly classified into:

1. **Symmetric Key Exchange**
2. **Asymmetric Key Exchange**

Symmetric Key Exchange

- In symmetric key exchange, both parties use the **same secret key** for encryption and decryption.
- The secret key must be securely shared between the communicating parties before communication begins.

Advantages:

- Fast encryption and decryption.
- Suitable for encrypting large amounts of data.

Limitation:

- Securely distributing the secret key can be difficult, especially when many users are involved.

Asymmetric Key Exchange

Asymmetric key exchange uses a pair of mathematically related keys:

- **Public Key** – Can be shared openly.
- **Private Key** – Kept secret by the owner.

It allows two parties to establish a common secret without directly transmitting the secret key over the communication channel.

3. Diffie-Hellman Algorithm

- The **Diffie-Hellman (DH) algorithm**, proposed by Whitfield Diffie and Martin Hellman, is a key agreement algorithm that allows two parties to establish a common shared secret over an insecure communication channel.
- The security of the algorithm is based on the difficulty of solving the **Discrete Logarithm Problem**.

Working of Diffie-Hellman

Let:

- P = A large prime number.
- G = A primitive root (generator) modulo P.
- a = Alice's private key.
- b = Bob's private key.

P and G are publicly known.

Step 1: Generate Private Keys

Alice selects a private key:

a = Alice's private key

Bob selects a private key:

b = Bob's private key

The private keys are kept secret.

Step 2: Generate Public Keys

Alice calculates her public key:

A = G^a mod P

Bob calculates his public key:

B = G^b mod P

The public keys can be exchanged over an insecure channel.

Step 3: Calculate Shared Secret Key

Alice uses Bob's public key:

K_a = B^a mod P

Bob uses Alice's public key:

K_β = A^b mod P

Since:

$K_a = K_\beta$

both Alice and Bob obtain the same shared secret key.

Step 4: Key Agreement

The shared secret key is not directly transmitted over the network.

Both parties calculate the same secret independently and can use it as a basis for symmetric encryption.

Algorithm

A. Diffie-Hellman Key Generation

1. Select a prime number P.
2. Select a primitive root G modulo P.
3. Select Alice's private key a.
4. Select Bob's private key b.
5. Calculate Alice's public key using:
   A = G^a mod P
6. Calculate Bob's public key using:
   B = G^b mod P
7. Exchange the public keys.
8. Alice calculates the shared secret:
   K_a = B^a mod P
9. Bob calculates the shared secret:
   K_β = A^b mod P
10. Compare K_a and K_β.
11. If both values are equal, the key exchange is successful.

Program: Diffie-Hellman Key Exchange

```
def diffie_hellman():

    P = int(input("Enter a prime number (P): "))
    G = int(input("Enter a primitive root (G): "))

    private_key_A = int(input("Enter Alice's private key (a): "))
    private_key_B = int(input("Enter Bob's private key (b): "))

    public_key_A = pow(G, private_key_A, P)
    public_key_B = pow(G, private_key_B, P)

    shared_secret_key_A = pow(public_key_B, private_key_A, P)
    shared_secret_key_B = pow(public_key_A, private_key_B, P)
```

```
    print("\nPublic Key for Alice (A):", public_key_A)
    print("Public Key for Bob (B):", public_key_B)

    print("\nShared Secret Key for Alice (Ka):",
          shared_secret_key_A)

    print("Shared Secret Key for Bob (Kb):",
          shared_secret_key_B)

    if shared_secret_key_A == shared_secret_key_B:
        print("\nKey Exchange Successful.")
        print("Both parties have the same shared secret key.")
    else:
        print("\nKey Exchange Failed.")


if __name__ == '__main__':
    diffie_hellman()
```

Sample Output

```
========== RESTART: C:/Users/CC2_50/Downloads/CIS_Prac5.py ========
Enter a prime number (p) : 11
Enter a primitive root (G) : 7
Enter Alice's private key (a) : 3
Enter Alice's private key (b) : 5

Public Key for Alice (a) : 2

Public Key for Bob (b) : 10

Shared Secret Key for Alice (Ka) :  10

Shared Secret Key for Bob (Kb) :  10
```